ETH zürich

D-PHYS, ETH Zürich

SciFi Detector Slow Control System

Giacomo Moretti

ETH Group Meeting

August 9th, 2026

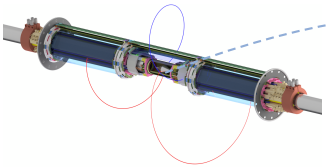
Context

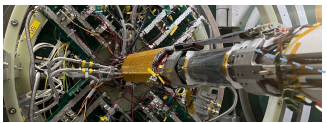
This is an account of the SciFi detector's slow control system: what got built, end to end, ahead of the data taking at PSI this October.

- The old slow control page, `Scifi-monitor.js`, grew organically over the years into something that worked – but was **hard to trust** and even harder to extend.
- So it was **replaced end to end**: a proper set of **Python scripts** driving the detector, and a new **custom operations page** a shifter can actually rely on.
- What follows is the complete picture – **installation**, the **interface** itself, **configuration**, **calibration**, and how results get **saved**.



Where we were → where we are







New page + full slow control

Saving

Calibration

Configuration

Power

User Interface

Installation

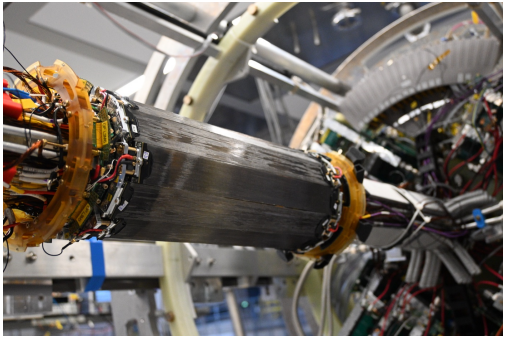
No control system

From an **ad-hoc JavaScript page** and a handful of scripts to a **complete slow control system** – built one floor at a time, this is the path the rest of the talk follows.

Installation



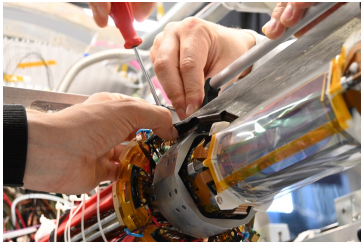
A week of installation at PSI



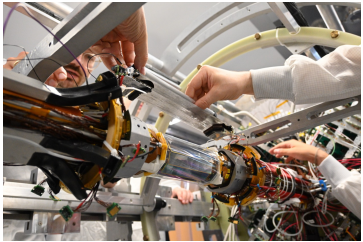


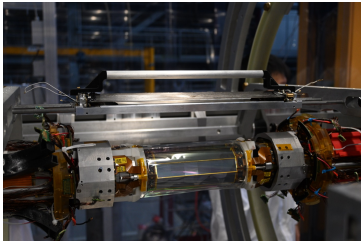
A full week at PSI went into physically **installing** and characterizing the **entire detector**, ring by ring, module by module – every piece of modularity the **Power** tab shows today (both rings, every super-module, every fibre and cable, **LV**, **HV**, **FEBs**, **readout**) was mechanically mounted, wired and verified, not just one part of it. The whole chain was characterized end to end, and it all works.

Installation, in pictures

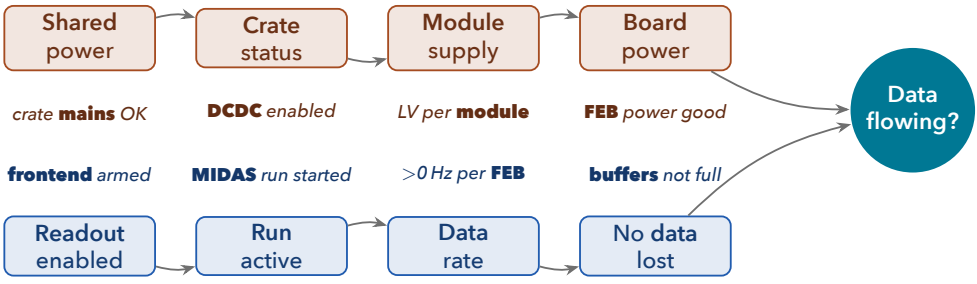








Two chains, one question



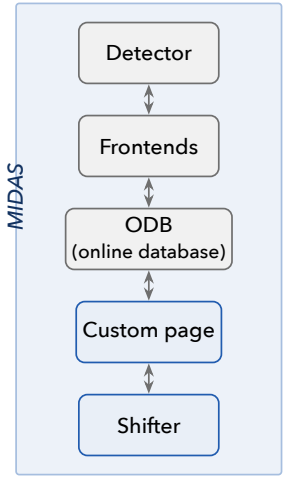
Two chains, checked separately – **power** can be all green while **readout** stays idle. Only when both agree does data actually flow.

User Interface

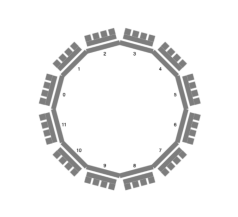


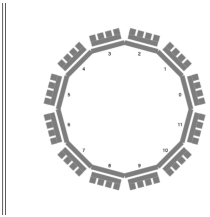
From hardware to shifter

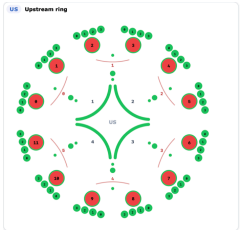
- The chain runs both ways: **hardware status** flows up to the shifter, and the **shifter's changes** flow back down to hardware, though not always instantly – some changes only apply at specific trigger points (e.g. a run start).
- **Nothing is ever hard-blocked**: every action stays clickable, and the page warns you instead of stopping you.
- **MIDAS** is the shared framework tying it together – frontends, ODB and this page all run inside it.

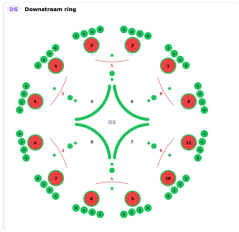


The page: then and now









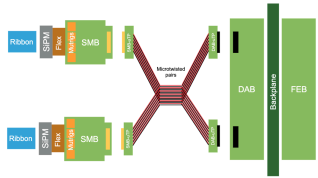
- The old page showed a static, unlabeled ring – **no live status**, nothing clickable.
- The new page splits in two: **Geometry** is pure diagnostic, read-only; **Power** / **Readout** / **Calibration** is the only editable surface.
- Configuring a value writes it to **ODB** – a frontend picks it up and pushes it onto the physical hardware.
- Real safety logic stays in **Python**, on purpose – never duplicated in JavaScript.

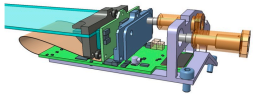
Power

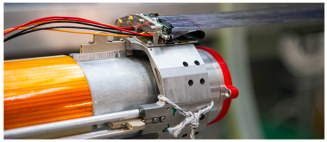


The SciFi detector, briefly

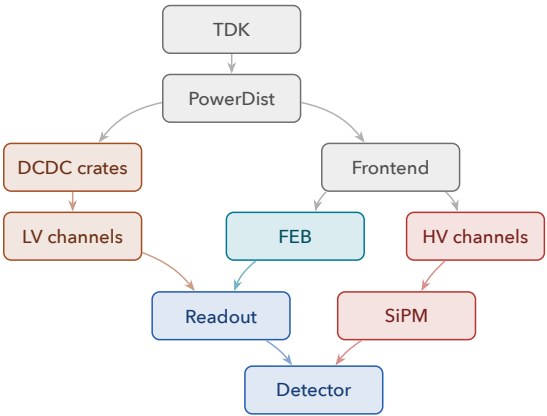
- **2 rings**, US / DS – 12 modules each; **24 SMB** (12+12) merge into 12 super-modules via a passive DAB.
- **12 FEB** (1:1 with LV); **96 ASIC** (32 ch. each) = **3072 channels**; **24 HV channels** (1:1 with SMB).
- Each module: {side, ring position, readout end} – the same key used across **ODB**, **geometry**, and **calibration**.
- SMB and DAB connect via microtwisted-pair links, carrying the raw **MuTRiG3** data downstream.







From detector to Power and Readout

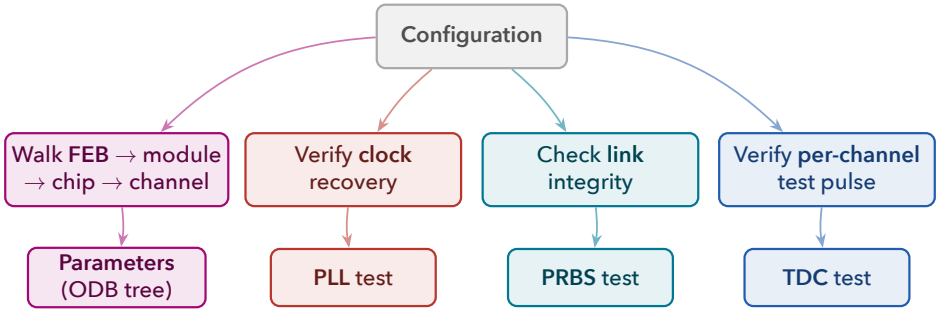


- TDK and PowerDist – the same first two devices for all three chains.
- LV – through its own DCDC crates – feeds the Readout.
- HV and FEB both wait for their Frontend.
- HV feeds the SiPM; FEB feeds the Readout too.
- Readout and SiPM both lead back to the detector.

Configuration

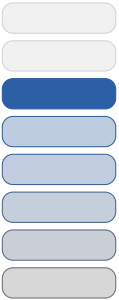


Configuration: one hub, four jobs

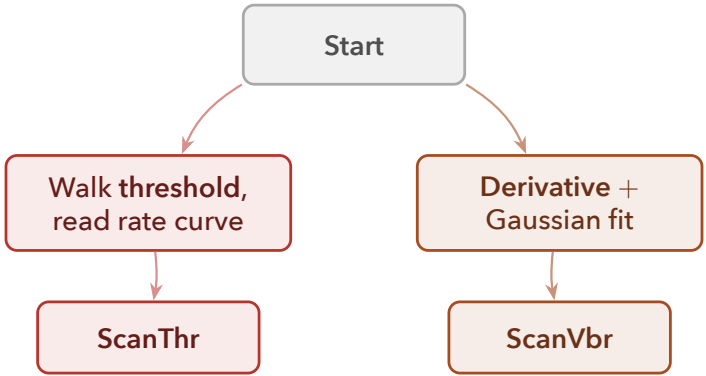


Configuration has grown beyond parameter editing into the detector's general hardware test hub – the old **Readout** tab folds in here entirely, and every parameter still writes to a dedicated **ODB** tree, read directly by the Python scripts.

Calibration



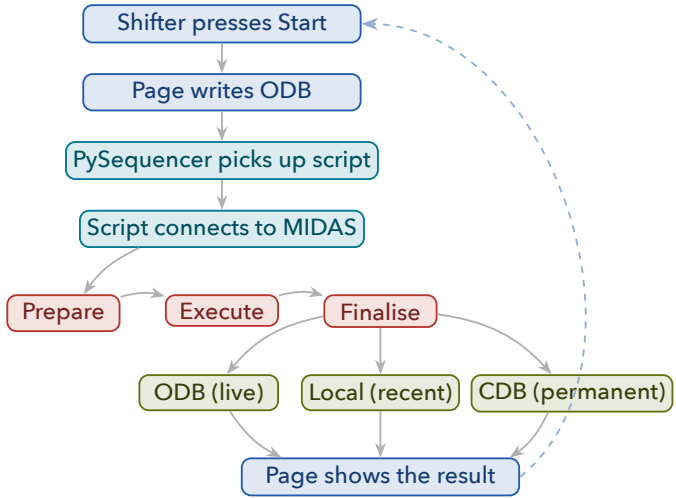
Two calibrations, one starting point



Both share the same `CalibrationBase` lifecycle – **prepare**, **execute**, **finalise**.

How a calibration actually runs

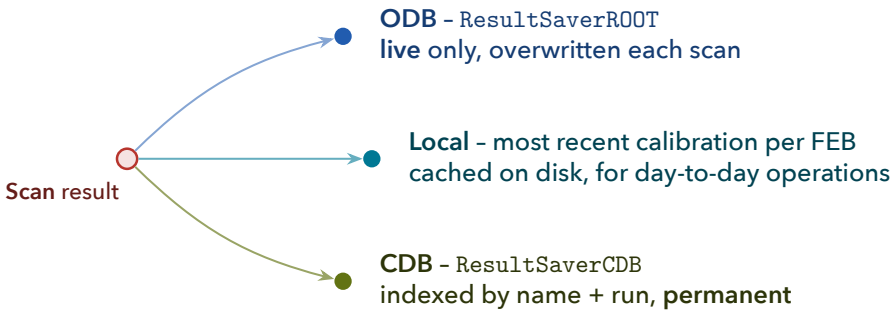
- **Start** – the page writes ODB and hands off to PySequencer.
- **Script** – PySequencer launches it, it connects to MIDAS.
- **Lifecycle** – prepare, execute, finalise.
- **Save** – result written to ODB, local, and CDB.
- **Result** – the page shows it back to the shifter.



Saving



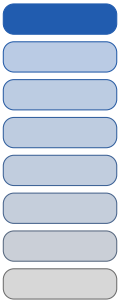
Saving: ODB, local, and CDB



All three destinations are genuinely wired into the calibration scripts now.

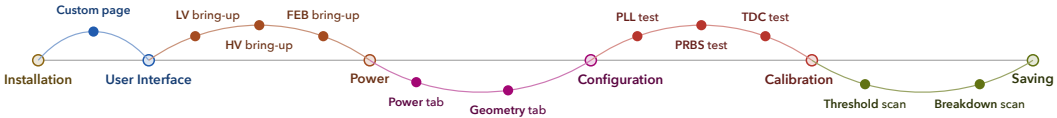
- **ODB** – *"What's the calibration right now?"*
- **Local** – *"What did we last calibrate, for operations?"*
- **CDB** – *"What was it on a given run?"*

Big Picture



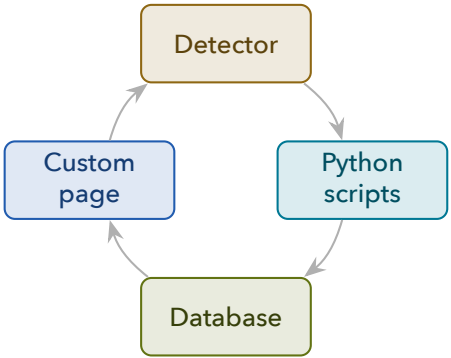
The work, as a branch diagram

It helps to think about this effort the way you'd think about a software repository.



There's a main line – the real operational workflow, installation through to saving – and every concrete piece of work is a branch off that line, sometimes several at once: three independent bring-up sequences, a family of hardware tests, two independent calibration scans. Each one gets built and tested on its own, then merged back once it's solid. None of these pieces are useful in isolation; the point was always to converge onto one coherent system.

How it all fits together



- **Detector** – matters once a script can drive it.
- **Python scripts** – matter once the result is saved.
- **Database** – matters once the page shows it back.
- **Custom page** – matters once a shifter can act on it.

This is a slow control system: no single piece works alone. Every step in this loop matters equally.

What this leaves behind

- **Result saving** is fully implemented and wired – the one piece that had to be ready for the data taking.
- The **shared code library** now properly covers SciFi: a real, reusable slow control foundation, not a one-off.
- The **custom page** is fully implemented – Power, Configuration and Calibration all wired to the real detector.
- The full **operational workflow** is implemented end to end: shifter action → ODB → Python script → result saved and shown back.

Installed and working on `mu3ebe`: LV, HV, FEBs, Readout

What's still open

Everything is installed, wired, and ready. The one thing left is a full end-to-end workflow test, including a real calibration run, on `mu3ebe`.

ETH zürich

Giacomo Moretti

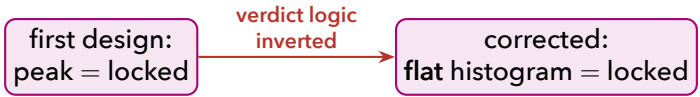
Mu3e Experiment
Scifi Detector Group
ETH Zürich
PSI, Villigen



Backup

PLL test: getting the verdict right

The PLL test is a good example of why it matters to check assumptions against real data, not just against the code.



The first version of the acceptance logic assumed a locked PLL would show up as a clear peak in the timing histogram – a reasonable-sounding guess. It turned out to be backwards: a genuinely locked PLL produces a **flat** histogram. Resolved directly with **Yifen Wan**, who owns the histogram analysis.

Geometry tab: drill down, three clicks deep

SM health dot

adds HV/Readout/
Calibration status

Module dot

expands the
32-channel grid

ASIC dot

US/DS are drawn as mirror rings with opposite rotation. Each click level is **cumulative**, not a page switch.





What's hardcoded vs. what's live

SAFE	Compile-time constants
SELF-HEALING	Name-pattern filters (`"SMB_"`, `"SciFi CH"`)
REAL RISK	Curated lists: `SciFi_Programs.js` (7), `SciFi_Alarms.js` (10), `HV_BOARDS` (32-cell board↔module map)
LIVE	Everything else

`HV_BOARDS` is the single biggest hardcoded piece on the whole page – flagged, not yet ported into Python/ODB.

Calibration entry points – a deliberate simplification

`thr_run.py` / `vbr_run.py` / `pll_run.py` bypass SciFi's own `CalibrationFactory` (never wired to a real entry point) and instead:

- build `Detector` + `Calibration` **directly**
- open their **own** `MidasClient`, with `try/finally: disconnect()`

Judged safer than pixel's own pattern

Pixel passes PySequencer's `seq` object in as the client – never independently verified. SciFi's explicit connect/disconnect is a stronger guarantee.

From design to a real save

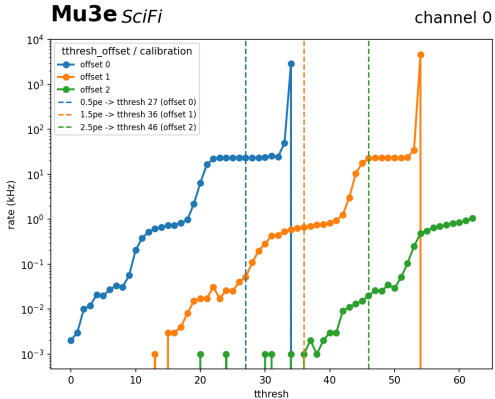
`.root` layout

5 trees – `Meta`, `RawScan`, `Results`, `Conditions`, `ChannelMasks` – the shape was restructured once per-module/per-ASIC parameters were properly pinned down.

`ResultSaverROOT` (per-tree upsert by key), `ResultSaverCDB` (opt-in POST to `mu3edb0/detcal`), and a local cache of the most recent per-FEB calibration are implemented and wired, reconciled with a colleague's parallel branch.

Calibration: threshold scan

- **ScanThr** finds the right discrimination point on every channel.
- Walks the **threshold** up per channel, reading the rate curve to find the crossing points at 0.5, 1.5 and 2.5 p.e.
- Runs independently per **FEB** – any subset of the detector can be (re-)calibrated on its own, and the result is what actually gets loaded as the operating threshold.



Calibration: IV scan

- **ScanVbr** locates each SiPM's breakdown voltage.
- Takes the **derivative** of the IV curve and fits a **Gaussian** to its peak – the fit mean μ gives V_{br}.
- Runs independently per **FEB** – the result directly sets each SiPM's HV operating point (overvoltage), a real physics constant, not just a diagnostic.

